

CAL Computer Science Club

L4 Track Model & UI State

用类型安全的 Track 模型与显式状态替代零散变量。

Flutter · Music Library Course

Recap: Music Library UI

- L3: 曲目行由 Track 数据构成
- 搜索词是暂时的页面状态
- 本地音频与曲目信息需要分开保存

上一课的 Widget 知识，将成为本课页面的起点。

多个平行 List 和一个 status String，很容易让曲名、文件与下载状态互相错位。

现在

一个 Widget 中混合界面、状态与数据操作



本课目标

用类型安全的 Track 模型与显式状态替代零散变量。

想一想：用户下一步最需要看到什么？

Track Model

Track 数据模型

把描述一首曲目的字段放进一个对象，并使用稳定 ID 关联记录。

- id: 本地唯一标识
- title / artist: 展示信息
- localPath: 音频文件位置
- sourceNote: 来源与授权记录



```
class Track {  
  const Track({  
    required this.id,  
    required this.title,  
    required this.artist,  
    required this.localPath,  
  });  
  
  final String id;  
  final String title;  
  final String artist;  
  final String localPath;  
}
```

final

对象创建后字段不再改变，减少意外修改。

数据与界面状态

Track 描述已保存内容；页面状态描述当前操作进度。

- Track 可写入本地数据库
- DownloadState 只表示导入流程
- 进度值属于 downloading 状态
- 错误状态要保留可读信息

● ● ●

```
sealed class ImportState {}  
class Idle extends ImportState {}  
class Importing extends ImportState {  
    Importing(this.progress);  
    final double progress;  
}  
class Imported extends ImportState {  
    Imported(this.track);  
    final Track track;  
}
```

不同状态

状态携带所需数据，避免
多组 bool 彼此矛盾。

一次导入如何更新界面

异步操作开始、成功或失败时，更新页面状态并重建界面。

- idle → importing(progress)
- 导入并保存成功 → imported(track)
- 网络、磁盘或授权检查失败 → failed(message)
- 取消操作也要回到可继续操作的状态

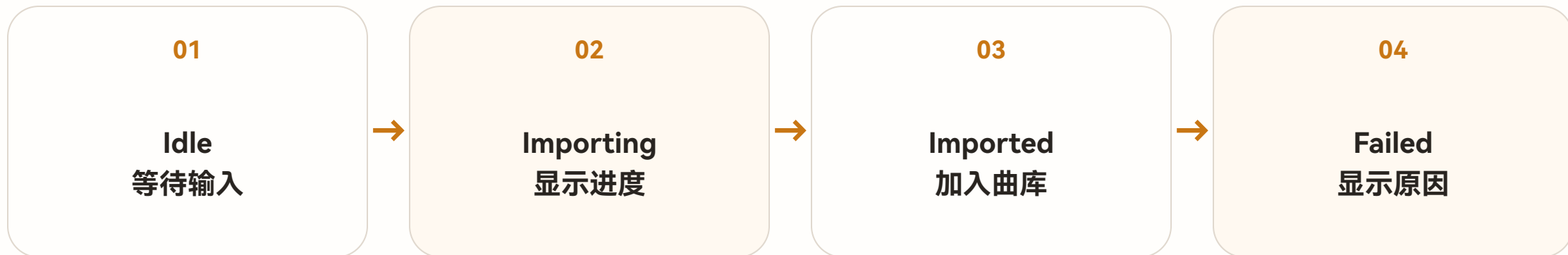
• • •

```
setState(() {  
  state = const Importing(0);  
});  
  
final track = await repository.import(uri);  
setState(() {  
  state = Imported(track);  
});
```

setState()

告诉 Flutter：状态变了，
请重新 build。

状态流转



用类型安全的 Track 模型与显式状态替代零散变量。

建立 Track 模型

从零散字段迁移到单个对象。先让曲目列表只接收 `List<Track>`。

- 1. 建立 `track.dart` 并定义不可变字段。
- 2. 创建两条 class sample Track。
- 3. 改造 `TrackRow`，让它只接收一个 `Track`。

跟做并运行

• • •

```
final sampleTracks = [
  Track(
    id: "class-01",
    title: "Rain Window",
    artist: "Class Sample",
    localPath: "/music/rain.mp3",
  ),
];

TrackRow(track: sampleTracks.first)
```

跟做检查

不使用互相冲突的
`isLoading / isDone /`
`hasError` 组合；每种状态都有
清楚的 UI 表现。

In-class Exercise

为导入界面设计状态类型。

1. 列出等待、进行中、成功、失败四种状态。
2. 指出进行中需要携带哪个值。
3. 说明成功状态如何把 Track 加入页面列表。

完成标准

不使用互相冲突的 `isLoading` / `isDone` / `hasError` 组合；每种状态都有清楚的 UI 表现。

独立完成 2 分钟，再与同伴对照检查。

Reference & Resources

课后可从这些官方文档继续查阅 API 与平台行为。

1. <https://docs.flutter.dev/data-and-backend/state-mgmt/intro>
2. <https://docs.flutter.dev/app-architecture/guide>

课程样本仅用于明确授权的音频；实际项目需再次核对平台条款与来源许可。